

AMARA Dating App — Official Backend API Specification

Document Version: 2.1 (PRD v1.1 Traceable Backend Specification)
PRD Reference: AMARA-RD-002 (Enhanced with Monetization & Subscriptions)
Target Audience: Backend Engineering Team, Frontend Engineering Team, DevOps, QA

1. Architecture Overview & Global Standards

This document establishes the single source of truth for all REST API endpoints required by the AMARA mobile frontend client. Every request payload, response schema, and status code defined here is guaranteed to match the frontend implementation.

1.1 Base URLs & Environments

- Production:** `https://api.amara-app.com/api/v1`
- Staging / Development:** `http://192.168.1.100:9395` (or local backend host)
- API Client:** Axios with automatic JWT Bearer token attachment and response interceptor.

1.2 Authentication & Security

- Authentication Scheme:** HTTP Bearer Token

Authorization: Bearer <JWT_ACCESS_TOKEN>

- All endpoints require authentication **except**:
- `POST /register`
- `POST /verify-register/otp`
- `POST /login`
- `POST /forgot-password/send-otp`
- `POST /forgot-password/reset`
- `POST /razorpay/webhook`

1.3 Standard Response Formats

Success Envelope (200 OK / 201 Created)

```
{
  "status": "success",
  "message": "Operation completed successfully",
  "data": { ... }
}
```

(Note: Some entity queries return the entity directly or as an array, as specified per endpoint below.)

Error Envelope (400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict)

```
{
  "status": "error",
  "message": "Human-readable explanation of error.",
  "details": "Technical explanation or validation details (optional)."
```

1.4 User Identifier Conventions

- All user entities have a unique string `userId` (e.g., `"usr_man_101"`, `"usr_woman_201"`).
- The backend must accept and return string identifiers for both `userId` and `id` to avoid 32-bit integer truncation.

2. Authentication & Account Lifecycle

Testing & Development Environment Note: Test OTP

- Non-Production Environments (Local / Staging / QA):**

- Third-party SMS gateways (Twilio, MSG91, Firebase) do not need to be triggered during local development and QA testing to avoid billing costs.
- The backend should accept the static test code **123456** as a valid OTP for:
 1. **Registration OTP Verification:** **POST /verify-register/otp**
 2. **Forgot Password Reset:** **POST /forgot-password/reset**
- In development mode (**NODE_ENV !== 'production'**), the backend can log the OTP to server console logs or optionally return **"debugOtp": "123456"** in the response.
- **Production Environment:**
 - In production (**NODE_ENV === 'production'**), the static test OTP **123456** is strictly disabled.
 - A real SMS gateway dispatches the random 6-digit OTP.
 - The OTP must never be returned in HTTP response bodies or logged in client-facing output.

2.1 Initiate Registration

- **Endpoint:** **POST /register**
- **Access:** Public
- **Description:** Validates mobile number (10+ digits), validates password match, triggers SMS OTP.
- **Request Body:**

```
{
  "name": "Rahul Sharma",
  "mobile": "9876543210",
  "password": "Password@123",
  "confirmPassword": "Password@123"
}
```

- **Success Response (200 OK):**

```
{
  "status": "success",
  "message": "Registration initiated. OTP sent to mobile.",
  "sessionId": "sess_9876543210_1727395200"
}
```

- **Error Responses:**
 - **400 Bad Request:** { "status": "error", "message": "Mobile number must be at least 10 digits." }
 - **400 Bad Request:** { "status": "error", "message": "Passwords do not match." }
 - **409 Conflict:** { "status": "error", "message": "Mobile number is already registered." }

2.2 Verify Registration OTP & Complete Account Creation

- **Endpoint:** **POST /verify-register/otp**
- **Access:** Public
- **Description:** Verifies OTP. On success, creates the user record and issues JWT token.
- **Request Body:**

```
{
  "mobile": "9876543210",
  "otp": "123456",
  "sessionId": "sess_9876543210_1727395200"
}
```

- **Success Response (200 OK):**

```
{
  "status": "success",
  "message": "User registered successfully",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "userId": "usr_man_101",
  "user": {
    "id": 1,
    "userId": "usr_man_101",
    "name": "Rahul Sharma",
    "displayName": "Rahul Sharma",
    "mobile": "9876543210",
    "gender": "man",
    "telegramUsername": ""
  }
}
```

- **Error Response (400 Bad Request):**

```
{
  "status": "error",
  "message": "Invalid OTP. Please check and try again."
}
```

2.3 User Login

- **Endpoint:** **POST** `/login`
- **Access:** Public
- **Description:** Authenticates existing user by mobile and password.
- **Request Body:**

```
{
  "mobile": "9876543210",
  "password": "Password@123"
}
```

- **Success Response (200 OK):**

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "userId": "usr_man_101",
  "user": {
    "id": 1,
    "userId": "usr_man_101",
    "name": "Rahul Sharma",
    "displayName": "Rahul Sharma",
    "mobile": "9876543210",
    "gender": "man",
    "telegramUsername": "rahul_amara"
  }
}
```

- **Error Responses:**
- **400 Bad Request:** { "status": "error", "message": "Mobile and password are required." }
- **401 Unauthorized:** { "status": "error", "message": "Invalid credentials. Please verify your password." }
- **404 Not Found:** { "status": "error", "message": "Account not found. Please register first." }

2.4 Forgot Password — Send Reset OTP

- **Endpoint:** **POST** `/forgot-password/send-otp`
- **Access:** Public
- **Request Body:**

```
{
  "mobile": "9876543210"
}
```

- **Success Response (200 OK):**

```
{
  "status": "success",
  "message": "Password reset OTP sent successfully",
  "sessionId": "reset_sess_9876543210"
}
```

2.5 Forgot Password — Reset Password

- **Endpoint:** **POST** `/forgot-password/reset`
- **Access:** Public
- **Request Body:**

```
{
  "mobile": "9876543210",
  "otp": "123456",
  "newPassword": "NewPassword@123"
}
```

- **Success Response (200 OK):**

```
{
  "status": "success",
  "message": "Password reset successfully. You can now login with your new password."
}
```

2.6 Account Deletion (FR-08, Screen #13)

- **Endpoint:** **DELETE** `/account/delete?userId={userId}`
- **Headers:** **Authorization:** Bearer `<token>`
- **Description:** Permanently deletes the user's account, profile, photos, and match history after two explicit user confirmations in the UI.

- **Success Response (200 OK):**

```
{
  "status": "success",
  "message": "Account deleted successfully."
}
```

■ 3. Profile & Media Management

3.1 Get Profile (Current User or Target Profile)

- **Endpoint:** **GET** `/profile/me/{userId}` or **POST** `/profile/me?userId={userId}`
- **Headers:** **Authorization:** **Bearer** `<token>`
- **Description:** Returns complete user profile information. Note: Sensitive fields (phone number, raw password) are NEVER returned.
- **Success Response (200 OK):**

```
{
  "id": 1,
  "userId": "usr_man_101",
  "name": "Rahul Sharma",
  "displayName": "Rahul, 25",
  "age": 25,
  "dob": "2001-05-10",
  "gender": "man",
  "bio": "Software engineer and amateur guitarist. Love fitness, indie cinema, and road trips.",
  "photos": [
    "https://images.unsplash.com/photo-1500648767791-00dcc994a43e?w=800&auto=format&fit=crop&q=80",
    "https://images.unsplash.com/photo-1506794778202-cad84cf45f1d?w=800&auto=format&fit=crop&q=80"
  ],
  "images": [
    "https://images.unsplash.com/photo-1500648767791-00dcc994a43e?w=800&auto=format&fit=crop&q=80",
    "https://images.unsplash.com/photo-1506794778202-cad84cf45f1d?w=800&auto=format&fit=crop&q=80"
  ],
  "telegramUsername": "rahul_amara"
}
```

3.2 Initial Profile Setup

- **Endpoint:** **POST** `/profile/{userId}/setup`
- **Headers:** **Authorization:** **Bearer** `<token>`
- **Description:** Called during user onboarding. Validates age ≥ 18 , 2–5 photos, bio (max 500 chars).
- **Request Body:** The frontend sends **multipart/form-data** so the endpoint must accept the profile fields as form fields and an optional **photo** file. For JSON-only clients, the same fields may be sent as JSON. The current frontend also sends **dto={JSON.stringify(profileFields)}** as a query parameter for compatibility; the backend should prefer the parsed form fields and support the **dto** fallback during integration.
- **Canonical profile fields:**

```
{
  "name": "Rahul Sharma",
  "displayName": "Rahul, 25",
  "dob": "2001-05-10",
  "gender": "man",
  "bio": "Software engineer and amateur guitarist. Love fitness and road trips.",
  "photos": [
    "https://images.unsplash.com/photo-1500648767791-00dcc994a43e?w=800&auto=format&fit=crop&q=80",
    "https://images.unsplash.com/photo-1506794778202-cad84cf45f1d?w=800&auto=format&fit=crop&q=80"
  ],
  "telegramUsername": "rahul_amara"
}
```

- **Success Response (200 OK):**

```
{
  "status": "success",
  "message": "Profile setup completed",
  "profile": {
    "userId": "usr_man_101",
    "name": "Rahul Sharma",
    "displayName": "Rahul, 25",
    "age": 25,
    "gender": "man",
    "photos": [
      "https://images.unsplash.com/photo-1500648767791-00dcc994a43e?w=800&auto=format&fit=crop&q=80",
      "https://images.unsplash.com/photo-1506794778202-cad84cf45f1d?w=800&auto=format&fit=crop&q=80"
    ],
    "telegramUsername": "rahul_amara"
  }
}
```

3.3 Profile Update Endpoints (PDF Endpoint #6)

[TIP] **Recommended Unified Endpoint:** Backend developers can implement a single endpoint **PUT /profile/update** (or **PUT /profile**) that accepts partial updates of any profile attributes (basic info, lifestyle details, dating preferences, photo list). The frontend automatically routes updates to **PUT /profile/update**.

3.3.0 Unified Profile Update (Recommended Single Endpoint)

- **Endpoint:** **PUT /profile/update** (Alias: **PUT /profile**)
- **Headers:** **Authorization: Bearer <token>**
- **Description:** Updates any combination of basic identity, lifestyle attributes, and dating preferences in a single call.
- **Request Body** (Any subset of fields):

```
{
  "userId": "usr_man_101",
  "displayName": "Rahul, 25",
  "name": "Rahul Sharma",
  "bio": "Software engineer and amateur guitarist.",
  "dob": "2001-05-10",
  "height": 178,
  "appearance": "Athletic",
  "bodyType": "Fit",
  "smoke": "Never",
  "drink": "Socially",
  "lookingFor": "Long-term relationship",
  "photos": [
    "https://images.unsplash.com/photo-1500648767791-00dcc994a43e?w=800",
    "https://images.unsplash.com/photo-1506794778202-cad84cf45f1d?w=800"
  ]
}
```

- **Success Response (200 OK):**

```
{
  "status": "success",
  "message": "Profile updated successfully",
  "data": {
    "userId": "usr_man_101",
    "displayName": "Rahul, 25",
    "bio": "Software engineer and amateur guitarist.",
    "height": 178,
    "bodyType": "Fit",
    "photos": [
      "https://images.unsplash.com/photo-1500648767791-00dcc994a43e?w=800",
      "https://images.unsplash.com/photo-1506794778202-cad84cf45f1d?w=800"
    ]
  }
}
```

3.3.1 Update Basic Profile (Legacy Endpoint)

- **Endpoint:** **PUT /profile/update-basic**
- **Headers:** **Authorization: Bearer <token>**
- **Description:** Updates basic identity info: Display Name, Bio, and Date of Birth / Age.
- **Request Body:**

```
{
  "userId": "usr_man_101",
  "displayName": "Rahul, 25",
  "name": "Rahul Sharma",
  "bio": "Software engineer and amateur guitarist. Love fitness, indie cinema, and road trips.",
  "dob": "2001-05-10",
  "age": 25
}
```

- **Success Response (200 OK):**

```
{
  "status": "success",
  "message": "Profile updated successfully",
  "data": {
    "userId": "usr_man_101",
    "displayName": "Rahul, 25",
    "name": "Rahul Sharma",
    "bio": "Software engineer and amateur guitarist. Love fitness, indie cinema, and road trips.",
    "dob": "2001-05-10",
    "age": 25
  }
}
```

3.3.2 Update Profile Details (More Info Attributes)

- **Endpoint:** **PUT** `/profile/update-details`
- **Headers:** **Authorization:** **Bearer** `<token>`
- **Description:** Updates user physical and personal attributes (Height, Appearance, Body Type, Languages, Ethnicity, English level, Kids, Net Worth, Bio, Telegram).
- **Request Body:**

```
{
  "userId": "usr_woman_206",
  "height": 175,
  "appearance": "Natural",
  "bodyType": "Average",
  "language": "English, Japanese",
  "englishLevel": "intermediate",
  "ethnicity": "Asian",
  "kidCount": "No kids",
  "netWorth": "Prefer not to say",
  "gender": "woman",
  "bio": "Looking for genuine connection and conversation.",
  "telegramUsername": "ananya_amara"
}
```

- **Success Response (200 OK):**

```
{
  "status": "success",
  "message": "Profile details updated successfully",
  "data": {
    "userId": "usr_woman_206",
    "height": 175,
    "appearance": "Natural",
    "bodyType": "Average",
    "language": "English, Japanese",
    "englishLevel": "intermediate",
    "ethnicity": "Asian",
    "kidCount": "No kids",
    "netWorth": "Prefer not to say"
  }
}
```

3.3.3 Update Profile Preferences (Lifestyle & Dating Goals)

- **Endpoint:** **PUT** `/profile/update-preferences`
- **Headers:** **Authorization:** **Bearer** `<token>`
- **Description:** Updates dating lifestyle preferences (Looking For, Smoking, Drinking habits, Ethnicity preferences).
- **Request Body:**

```
{
  "userId": "usr_woman_206",
  "lookingFor": "Long-term, Marriage",
  "smoke": "Never",
  "drink": "Socially",
  "ethnicity": "Asian"
}
```

- **Success Response (200 OK):**

```
{
  "status": "success",
  "message": "Profile preferences updated successfully",
  "data": {
    "userId": "usr_woman_206",
    "lookingFor": "Long-term, Marriage",
    "smoke": "Never",
    "drink": "Socially",
    "ethnicity": "Asian"
  }
}
```

3.4 Photo Gallery & Uploads

- **Upload Image:** **POST** `/profile/upload-image`
- **Headers:** **Content-Type:** `multipart/form-data`
- **Form Fields:** **userId** (string), **image** (binary file). The backend may also accept **userId** as a query parameter.
- **Success Response (200 OK):**

```
{
  "status": "success",
  "message": "Image uploaded successfully",
  "id": 105,
  "imageUrl": "https://storage.amara-app.com/photos/usr_man_101_105.jpg"
}
```

- **Get User Photos:** **GET** `/users/{userId}/images`
- **Success Response (200 OK):**

```
[
  {
    "id": 1,
    "userId": "usr_man_101",
    "imageUrl": "https://images.unsplash.com/photo-1500648767791-00dcc994a43e?w=800",
    "isProfile": true,
    "uploadedAt": "2026-09-26T12:00:00Z"
  },
  {
    "id": 2,
    "userId": "usr_man_101",
    "imageUrl": "https://images.unsplash.com/photo-1506794778202-cad84cf45f1d?w=800",
    "isProfile": false,
    "uploadedAt": "2026-09-26T12:05:00Z"
  }
]
```

- **Set Profile Picture:** **PUT** `/users/{userId}/profile-photo/{imageId}`
- **Success Response (200 OK):**

```
{
  "status": "success",
  "message": "Profile photo updated"
}
```

- **Delete Photo:** **DELETE** `/users/images/{imageId}`
- **Success Response (200 OK):**

```
{
  "status": "success",
  "message": "Image deleted"
}
```

- **Selfie Verification Upload:** **POST** `/profile/selfie/upload`
- **Headers:** **Content-Type:** `multipart/form-data`
- **Form Fields:** **userId** (string), **selfie** (selfie binary). Accept **image** as an alias for compatibility.
- **Success Response (200 OK):**

```
{
  "status": "success",
  "message": "Selfie uploaded (Verification pending)"
}
```

- **Selfie Verification Confirmation:** **PUT** `/profile/selfie/verify/{userId}`
- The frontend sends the authenticated user identifier in the path. A server may also expose **PUT** `/profile/selfie/verify` with **userId** in the body, but it must support the path form used by the client.
- **Success Response (200 OK):**

```
{
  "status": "success",
  "message": "Verified"
}
```

■ 4. Discovery & Browsing Feed (Men Only)

[!IMPORTANT] **Domain Rule BR-01 & BR-04:** Only male users browse profiles. Discovery endpoint (**/dashboard/recent**) **MUST** exclusively return **verified women profiles**. Phone numbers, passwords, and raw contact identifiers are strictly withheld from feed responses.

4.1 Recent Women Profiles

- **Endpoint:** **GET /dashboard/recent**
- **Query Parameters:** **page=0&size=10**
- **Headers:** **Authorization: Bearer <token>**
- **Success Response (200 OK):**

```
[
  {
    "id": 2,
    "userId": "usr_woman_201",
    "displayName": "Ananya, 24",
    "name": "Ananya Roy",
    "age": 24,
    "gender": "woman",
    "bio": "Architect by day, espresso enthusiast by night. Spontaneous adventures welcome!",
    "photos": [
      "https://images.unsplash.com/photo-1494790108377-be9c29b29330?w=800&auto=format&fit=crop&q=80",
      "https://images.unsplash.com/photo-1517841905240-472988babdf9?w=800&auto=format&fit=crop&q=80"
    ],
    "profileImageUrl": "https://images.unsplash.com/photo-1494790108377-be9c29b29330?w=800&auto=format&fit=crop&q=80"
  },
  {
    "id": 3,
    "userId": "usr_woman_202",
    "displayName": "Priya, 23",
    "name": "Priya Sen",
    "age": 23,
    "gender": "woman",
    "bio": "Classical dancer, literature lover, and passionate foodie exploring Bangalore.",
    "photos": [
      "https://images.unsplash.com/photo-1534528741775-53994a69daeb?w=800&auto=format&fit=crop&q=80"
    ],
    "profileImageUrl": "https://images.unsplash.com/photo-1534528741775-53994a69daeb?w=800&auto=format&fit=crop&q=80"
  }
]
```

■ 5. Dating Requests & Connections

[!IMPORTANT] **Domain Rule BR-01 & BR-02:** - Only men send requests (**POST /connections/send**). Women receive requests in their inbox (**GET /connections/received**). - Request status flow: **PENDING** → **APPROVED**. - When a woman approves a request via **PUT /connections/accept**, the status changes to **APPROVED**, unlocking mutual Telegram contacts.

5.1 Send Dating Request (Men Only)

- **Endpoint:** **POST /connections/send**
- **Headers:** **Authorization: Bearer <token>**
- **Validation Rules:**
 - **senderId** and **receiverId** are required.
 - Reject self-request (**senderId === receiverId**) with **400 Bad Request**.
 - Reject request from female sender or targeting male receiver with **403 Forbidden**.
 - Reject duplicate requests with **409 Conflict**.
- **Request Body:**

```
{
  "senderId": "usr_man_101",
  "receiverId": "usr_woman_201"
}
```

- **Success Response (201 Created):**

```
{
  "status": "success",
  "message": "Request Sent",
  "data": {
    "id": "req_101_201",
    "senderId": "usr_man_101",
    "receiverId": "usr_woman_201",
    "status": "PENDING",
    "createdAt": "2026-09-26T14:30:00Z",
    "updatedAt": "2026-09-26T14:30:00Z"
  }
}
```

- **Error Responses:**

- **400 Bad Request:** { "status": "error", "message": "You cannot send a dating request to yourself." }
- **403 Forbidden:** { "status": "error", "message": "Women cannot send dating requests per BR-01." }
- **409 Conflict:** { "status": "error", "message": "A request to this profile has already been sent." }

5.2 Get Sent Requests (Men Only)

- **Endpoint:** **GET** /connections/sent?userId={userId}
- **Headers:** **Authorization:** Bearer <token>
- **Description:** Returns all requests sent by the logged-in male user.
- **Success Response (200 OK):**

```
[
  {
    "id": "req_101_201",
    "senderId": "usr_man_101",
    "receiverId": "usr_woman_201",
    "status": "PENDING",
    "createdAt": "2026-09-26T14:30:00Z",
    "updatedAt": "2026-09-26T14:30:00Z",
    "receiver": {
      "userId": "usr_woman_201",
      "displayName": "Ananya, 24",
      "name": "Ananya Roy",
      "age": 24,
      "gender": "woman",
      "bio": "Architect by day, espresso enthusiast by night.",
      "photos": [
        "https://images.unsplash.com/photo-1494790108377-be9c29b29330?w=800"
      ],
      "telegramUsername": ""
    }
  }
]
```

(Privacy rule: for **PENDING**, both **sender.telegramUsername** and **receiver.telegramUsername** MUST be "" or omitted. For **APPROVED**, both usernames MUST be returned when the corresponding user has connected Telegram; otherwise the value MUST be "".)

5.3 Get Received Requests Inbox (Women Only)

- **Endpoint:** **GET** /connections/received?userId={userId}
- **Headers:** **Authorization:** Bearer <token>
- **Description:** Returns all incoming requests for the logged-in female user, ordered newest first.
- **Success Response (200 OK):**

```
[
  {
    "id": "req_101_201",
    "senderId": "usr_man_101",
    "receiverId": "usr_woman_201",
    "status": "PENDING",
    "createdAt": "2026-09-26T14:30:00Z",
    "updatedAt": "2026-09-26T14:30:00Z",
    "sender": {
      "userId": "usr_man_101",
      "displayName": "Rahul, 25",
      "name": "Rahul Sharma",
      "age": 25,
      "gender": "man",
      "bio": "Software engineer and amateur guitarist.",
      "photos": [
        "https://images.unsplash.com/photo-1500648767791-00dcc994a43e?w=800"
      ],
      "telegramUsername": ""
    }
  }
]
```

5.4 Approve Dating Request (Women Only)

- **Endpoint:** **PUT /connections/accept**
- **Headers:** **Authorization: Bearer <token>**
- **Request Body:**

```
{
  "requestId": "req_101_201",
  "userId": "usr_woman_201"
}
```

- **Success Response (200 OK):**

```
{
  "status": "success",
  "message": "Request APPROVED successfully. Telegram contact is now unlocked!",
  "data": {
    "id": "req_101_201",
    "senderId": "usr_man_101",
    "receiverId": "usr_woman_201",
    "status": "APPROVED",
    "updatedAt": "2026-09-26T15:00:00Z",
    "sender": {
      "userId": "usr_man_101",
      "displayName": "Rahul",
      "telegramUsername": "rahul_amara"
    },
    "receiver": {
      "userId": "usr_woman_201",
      "displayName": "Ananya",
      "telegramUsername": "ananya_amara"
    }
  }
}
```

- **Error Response (404 Not Found):**

```
{
  "status": "error",
  "message": "Connection request not found."
}
```

5.5 Get Approved Connections List

- **Endpoint:** **GET /connections/list?userId={userId}**
- **Headers:** **Authorization: Bearer <token>**
- **Description:** Returns all mutually approved connections for the user. Both partners' profiles are attached. Telegram usernames are revealed only because the request is **APPROVED**; an unconnected user has an empty username and the client **MUST** show **Telegram not connected**.
- **Success Response (200 OK):**

```
[
  {
    "id": "req_101_201",
    "senderId": "usr_man_101",
    "receiverId": "usr_woman_201",
    "status": "APPROVED",
    "createdAt": "2026-09-26T14:30:00Z",
    "updatedAt": "2026-09-26T15:00:00Z",
    "sender": {
      "userId": "usr_man_101",
      "displayName": "Rahul, 25",
      "name": "Rahul Sharma",
      "photos": ["https://images.unsplash.com/photo-1500648767791-00dcc994a43e?w=800"],
      "telegramUsername": "rahul_amara"
    },
    "receiver": {
      "userId": "usr_woman_201",
      "displayName": "Ananya, 24",
      "name": "Ananya Roy",
      "photos": ["https://images.unsplash.com/photo-1494790108377-be9c29b29330?w=800"],
      "telegramUsername": "ananya_amara"
    }
  }
]
```

5.6 Check Connection Status Between Two Users

- **Endpoint:** **GET** `/connections/status?user1={u1}&user2={u2}`
- **Headers:** **Authorization:** **Bearer** `<token>`
- **Success Response (200 OK):**

```
{
  "status": "APPROVED" // Values: "NONE" | "PENDING" | "APPROVED"
}
```

🚩 6. Telegram Contact Handoff

[!IMPORTANT] **Domain Rule BR-03:** The **GET** `/telegram/link` endpoint **MUST** return **HTTP 403 Forbidden** if the two users do not have an **APPROVED** connection.

6.1 Connect Telegram Username

- **Endpoint:** **POST** `/telegram/connect`
- **Headers:** **Authorization:** **Bearer** `<token>`
- **Request Body or Query Params** (the current frontend sends query parameters):

```
{
  "userId": "usr_man_101",
  "username": "rahul_amara"
}
```

- **Success Response (200 OK):**

```
{
  "status": "success",
  "message": "Telegram connected successfully",
  "telegramUsername": "rahul_amara"
}
```

- **Validation Error (400 Bad Request):**

```
{
  "status": "error",
  "message": "Telegram username is required."
}
```

- **Behavior:** The username is normalized by removing a leading `@`. The client must not send a placeholder such as **pending**; it must send the user's actual Telegram username.

6.2 Get Partner's Telegram Link (Post-Approval Only)

- **Endpoint:** **GET** `/telegram/link?userId={targetUserId}`
- **Headers:** **Authorization:** **Bearer** `<token>`
- **Success Response (200 OK):**

```
{
  "telegramUsername": "ananya_amara",
  "link": "https://t.me/ananya_amara"
}
```

- **Unconnected Partner Response (200 OK):**

```
{
  "telegramUsername": "",
  "link": ""
}
```

The UI must show **Telegram not connected** and must not open a placeholder bot link for a partner.

- **Error Response (403 Forbidden):**

```
{
  "status": "error",
  "message": "Telegram contact is locked until the connection request is approved."
}
```

6.3 Privacy Rules for Profile and Discovery Responses

The following endpoints return public or non-approved profile data and MUST NOT include a Telegram username:

- **GET /dashboard/recent**
- **GET /profile/me?userId={otherUserId}** when the requester has no **APPROVED** connection to that user

For those cases, the response must either omit **telegramUsername** or return it as an empty string. A Telegram username may be returned only for:

1. The authenticated user's own profile; or
2. A request/connection whose status is **APPROVED**.

Example pre-approval profile response:

```
{
  "userId": "usr_woman_201",
  "displayName": "Ananya",
  "gender": "woman",
  "telegramUsername": ""
}
```

Example approved connection response:

```
{
  "status": "APPROVED",
  "sender": { "userId": "usr_man_101", "telegramUsername": "rahul_amara" },
  "receiver": { "userId": "usr_woman_201", "telegramUsername": "ananya_amara" }
}
```

7. Monetization & Subscriptions (Razorpay)

7.1 Subscription Plans

Plan ID	Display Name	Duration	Price (INR)	Amount (Paise)	Key Features
BASIC	Standard	1 Month	■99	9900	10 Daily Requests, Standard Discovery
GOLD	Premium	3 Months	■199	19900	20 Daily Requests, Priority Feed, See who liked
PREMIUM	Elite	6 Months	■499	49900	Unlimited Requests, Top Priority, Verified Badge

7.2 Create Razorpay Order

- **Endpoint:** **POST /razorpay/create-order**
- **Headers:** **Authorization: Bearer <token>**
- **Request Body or Query Params** (the current frontend sends query parameters; the backend should accept either form):

```
{
  "userId": "usr_man_101",
  "plan": "GOLD"
}
```

- **Success Response (200 OK):**

```
{
  "orderId": "order_rzp_9876543210",
  "id": "order_rzp_9876543210",
  "amount": 19900,
  "currency": "INR",
  "key": "rzp_live_XXXXXXXXXX",
  "plan": "GOLD",
  "userId": "usr_man_101"
}
```

7.3 Verify Payment & Activate Subscription

- **Endpoint:** **POST** `/razorpay/verify`
- **Headers:** **Authorization:** **Bearer** `<token>`
- **Description:** Backend validates HMAC SHA256 signature using Razorpay Secret (**HMAC_SHA256**(`orderId + "|" + paymentId, SECRET`)).
- **Request Body:**

```
{
  "orderId": "order_rzp_9876543210",
  "paymentId": "pay_9876543210",
  "signature": "320a9a4b872c01928374e6f48392019485720193847201948372019485720193"
}
```

The authenticated user is derived from the Bearer token. **userId** and **plan** may be accepted as additional fields, but the backend must validate them against the Razorpay order and must never trust a client-supplied user ID for ownership.

- **Success Response (200 OK):**

```
{
  "status": "success",
  "message": "Payment verified and subscription activated successfully!",
  "subscription": {
    "id": "sub_101_01",
    "userId": "usr_man_101",
    "plan": "GOLD",
    "status": "ACTIVE",
    "startDate": "2026-09-26T00:00:00.000Z",
    "endDate": "2026-12-26T00:00:00.000Z",
    "amount": 199
  }
}
```

7.4 Get User Subscription Status

- **Endpoint:** **GET** `/subscriber/status?userId={userId}`
- **Headers:** **Authorization:** **Bearer** `<token>`
- **Success Response (200 OK - Active):**

```
{
  "active": true,
  "plan": "GOLD",
  "planType": "GOLD",
  "startDate": "2026-09-26T00:00:00.000Z",
  "endDate": "2026-12-26T00:00:00.000Z",
  "status": "ACTIVE"
}
```

- **Success Response (200 OK - Inactive/Expired):**

```
{
  "active": false,
  "plan": null,
  "startDate": null,
  "endDate": null,
  "status": "EXPIRED"
}
```

7.5 Get Remaining Subscription Days

- **Endpoint:** **GET** `/subscriber/remaining-days?userId={userId}`
- **Headers:** **Authorization:** **Bearer** `<token>`
- **Success Response (200 OK):**

7.6 Razorpay Webhook Handler

- **Endpoint:** **POST** `/razorpay/webhook`
- **Access:** Public (Verified via **X-Razorpay-Signature** webhook header)
- **Success Response (200 OK):**

```
{
  "status": "received"
}
```

8. Safety, Reports & Moderation (FR-28, Screen #14)

8.1 Submit Safety Report

- **Endpoint:** **POST** `/reports/report` (or **POST** `/report`)
- **Headers:** **Authorization:** **Bearer** `<token>`
- **Request Body:**

```
{
  "byUserId": "usr_man_101",
  "targetUserId": "usr_woman_202",
  "reason": "Inappropriate Content",
  "message": "User photo contains unapproved promotional material."
}
```

- **Success Response (200 OK):**

```
{
  "status": "success",
  "message": "Profile has been reported. Our safety team will review it shortly."
}
```

9. Database Seed Fixtures & Test Accounts

To immediately test the entire application locally or on staging, seed your backend database with these standard accounts:

9.1 Test Users Table

User ID	Name	Mobile	Password	Gender	Telegram Username	Role
usr_man_101	Rahul Sharma	9876543210	Password@123	man	rahul_amara	Male User (Browses & sends requests)
usr_woman_201	Ananya Roy	9876543211	Password@123	woman	ananya_amara	Female User (Receives & approves requests)
usr_woman_202	Priya Sen	9876543213	Password@123	woman	priya_amara	Female User (Receives & approves requests)
usr_man_103	Kabir Malhotra	9876543214	Password@123	man	kabir_amara	Male User (Active requester)
usr_man_104	Siddharth Verma	9876543215	Password@123	man	sid_amara	Male User

9.2 Default Requests Table

Request ID	Sender ID	Receiver ID	Status	Note
req_101_201	usr_man_101 (Rahul)	usr_woman_201 (Ananya)	PENDING	Test approval flow in Ananya's inbox
req_103_201	usr_man_103 (Kabir)	usr_woman_201 (Ananya)	PENDING	Test multi-request inbox in Ananya's inbox
req_101_202	usr_man_101 (Rahul)	usr_woman_202 (Priya)	APPROVED	Test approved connection & Telegram handoff

10. Core Data Entities & Field Requirements (PRD Section 7)

The backend team is free to use any database engine (PostgreSQL, MySQL, MongoDB) or ORM (Prisma, TypeORM, Hibernate, Django). There is no requirement to execute pre-defined SQL scripts. However, all persisted entities and API responses MUST satisfy these canonical data requirements:

10.1 Required Entity Schemas

Entity	Required Fields	Notes & Validation
User	id , userId (string), name , mobile , password , gender , telegramUsername , createdAt	userId must be returned as a unique string (e.g. usr_man_101). Password must be stored as a strong one-way hash.
Profile	userId , displayName , dob / age , bio , photos[] , gender , telegramUsername	age must be ≥ 18 . bio maximum 500 characters. photos array must have between 2 and 5 URLs.
Request	id , senderId , receiverId , status , createdAt , updatedAt	senderId (man), receiverId (woman). Status must only be PENDING or APPROVED .
Subscription	id , userId , plan , status , startDate , endDate , orderId , paymentId , amount , createdAt	plan is BASIC , GOLD , or PREMIUM . status is ACTIVE or EXPIRED . Amount in paise/rupees.

10.2 Global Data & Privacy Rules

- User Identifier Consistency:** Every user object returned to the app must include the string **userId**.
- Contact Privacy:** Profile and request responses must include **telegramUsername** only when the connection is **APPROVED** or for the authenticated user's own profile. For unapproved/public profiles, it must be empty ("").
- Sensitive Field Shielding:** No mobile number, password, password hash, or email may ever be exposed in browse, feed, or connection request responses.
- Subscription Normalization:** Subscription status must always return normalized **active** flag, **plan** type, and **endDate**.

11. PRD v1.1 Traceability & Final Backend Decisions

This section maps the attached **AMARA Dating App Requirements v1.1** to the specification above. It is normative: when an older example conflicts with this section, this section and the PRD take precedence.

11.1 Required domain state and privacy rules

The backend MUST enforce these rules server-side; the mobile client must not be trusted to enforce them:

PRD rule	Backend behavior
BR-01	POST /connections/send is allowed only when the authenticated sender is gender=man and the receiver is gender=woman . Otherwise return 403 . The home/discovery APIs are not available to women.
BR-02 / BR-24	A connection has only PENDING or APPROVED . There are no decline, reject, cancel, recall, or revert APIs. PUT /connections/accept is the only state transition.
BR-03 / FR-26	Do not return Telegram or any personal contact detail for a non-approved pair. GET /telegram/link returns 403 until the pair is APPROVED .
BR-04	Men receive women in discovery. Women receive only men who requested them in /connections/received .
BR-05 / FR-19	Enforce a unique (senderId , receiverId) pair and reject self-requests with 400 ; duplicates return 409 .
BR-06 / FR-08	Before browsing or sending, require display name, valid DOB, gender, bio, and at least 2 photos. Profile photos must be 2–5.
BR-07 / FR-29	Reject DOB values that make the user younger than 18 with 400 .
BR-08 / NFR-08	Never return mobile , password , passwordHash , OTP, payment secrets, or internal authentication data in another user's profile, discovery, request, or log output.
BR-09	Treat a subscription as active only when status=ACTIVE and endDate > now . Expired subscriptions must be returned as inactive/expired.
BR-10	Activate a subscription only after Razorpay signature verification succeeds.
BR-11	Return normalized plan capabilities in subscription status: dailyRequestLimit , priorityHandoff , and eliteBadge .

11.2 Canonical entity fields

All responses used by the app MUST use string identifiers and ISO-8601 UTC timestamps.

```
{
  "user": {
    "id": "1",
    "userId": "usr_man_101",
    "name": "Rahul Sharma",
    "gender": "man",
    "createdAt": "2026-09-26T12:00:00.000Z"
  },
  "profile": {
    "userId": "usr_man_101",
    "displayName": "Rahul Sharma",
    "dob": "2001-05-10",
    "age": 25,
    "gender": "man",
    "bio": "Software engineer and amateur guitarist.",
    "photos": ["https://cdn.example.com/profile/1.jpg", "https://cdn.example.com/profile/2.jpg"],
    "telegramUsername": ""
  },
  "request": {
    "id": "req_101_201",
    "senderId": "usr_man_101",
    "receiverId": "usr_woman_201",
    "status": "PENDING",
    "createdAt": "2026-09-26T14:30:00.000Z",
    "updatedAt": "2026-09-26T14:30:00.000Z"
  }
}
```

Profile validation:

- **displayName/name**: required, maximum 120 characters.
- **bio**: optional text, maximum 500 characters.
- **dob**: required for profile completion; age is calculated by the backend and must be at least 18.
- **gender**: required enum **man** | **woman**.
- **photos**: required for completion, minimum 2 and maximum 5.
- **telegramUsername**: optional; normalize a leading @ away and return an empty string when not connected.

11.3 OTP and session specification additions

OTP resend cooldown is a backend rule, not only a UI timer. **POST /register** and **POST /forgot-password/send-otp** MUST reject another send for 30 seconds for the same mobile/session.

Successful OTP responses SHOULD include the following fields so the client can render the cooldown consistently:

```
{
  "status": "success",
  "message": "A verification code has been sent to your mobile.",
  "sessionId": "sess_9876543210_1727395200",
  "cooldownSeconds": 30,
  "resendAvailableAt": "2026-09-26T12:00:30.000Z"
}
```

The production API MUST never return the OTP. A wrong, expired, reused, or mismatched OTP returns **400** with a readable message. An expired JWT returns **401**; the frontend clears the session and routes to Login.

Development & Test OTP Mode:

For local development, integration testing, and staging environments where SMS delivery is not enabled:

- The backend MUST accept **123456** as a valid OTP for any mobile number during registration (**POST /verify-register/otp**) and password reset (**POST /forgot-password/reset**).
- In non-production modes, the backend may additionally return **"debugOtp": "123456"** in the response payload or print it to the server console log for ease of end-to-end API testing.

11.4 Canonical endpoint matrix from the PRD

The following endpoints are release-scope and must be implemented with the request/response examples in Sections 2–8:

Capability	Method and endpoint	Required response
Register	POST /register	sessionId , cooldown fields; no OTP in production
Verify registration	POST /verify-register/otp	JWT token , string userId , sanitized user
Login	POST /login	JWT token , string userId , sanitized user , profile completion data when available
Forgot password	POST /forgot-password/send-otp , POST /forgot-password/reset	OTP session, then reset confirmation
My/target profile	GET /profile/me/{userId} or POST /profile/me?userId=	Profile, photos, privacy-safe Telegram field

Profile setup/edit	POST /profile/{userId}/setup, PUT /profile/update-basic, PUT /profile/update-details, PUT /profile/update-preferences	Updated profile
Photos	POST /profile/upload-image, GET /users/{userId}/images, PUT /users/{userId}/profile-photo/{imageId}, DELETE /users/images/{imageId}	Immediate image list/URL updates
Browse	GET /dashboard/recent	Women-only paginated profiles for men
Requests	POST /connections/send, GET /connections/sent, GET /connections/received, PUT /connections/accept, GET /connections/status	Request objects with only PENDING/APPROVED
Telegram	POST /telegram/connect, GET /telegram/link	Username/link only for owner or approved pair
Payments	POST /razorpay/create-order, POST /razorpay/verify, GET /subscriber/status, GET /subscriber/remaining-days	Razorpay order/payment verification and normalized subscription
Safety	POST /reports/report	Report confirmation; reason required
Account	DELETE /account/delete	Confirmation-safe two-step account deletion

11.5 Subscription capabilities and payment retry specification

The product names and prices are fixed by FR-30:

Plan ID	User-facing name	Duration	Price	Current client capability response
BASIC	Standard	1 month	INR 99	dailyRequestLimit: 10, priorityHandoff: false, eliteBadge: false
GOLD	Premium	3 months	INR 199	dailyRequestLimit: 20, priorityHandoff: true, eliteBadge: false
PREMIUM	Elite	6 months	INR 499	dailyRequestLimit: null (unlimited), priorityHandoff: true, eliteBadge: true

GET /subscriber/status MUST return:

```
{
  "active": true,
  "plan": "PREMIUM",
  "planType": "PREMIUM",
  "status": "ACTIVE",
  "startDate": "2026-09-26T00:00:00.000Z",
  "endDate": "2027-03-26T00:00:00.000Z",
  "dailyRequestLimit": null,
  "priorityHandoff": true,
  "eliteBadge": true,
  "remainingDays": 180
}
```

POST /razorpay/verify MUST accept **orderId**, **paymentId**, **signature**, **plan**, and authenticated **userId**. It MUST return **400** or **402** without activating the plan when verification fails. Recommended error codes:

```
{ "status": "error", "code": "PAYMENT_VERIFICATION_FAILED", "message": "We could not confirm th
is payment. Please try again." }
```

```
{ "status": "error", "code": "PAYMENT_CANCELLED", "message": "Payment was cancelled. Your plan
was not changed." }
```

The client remains on the subscription modal and may retry. Razorpay handles PCI-DSS-sensitive payment data; the AMARA backend and mobile app must not store card, UPI, CVV, or bank credentials.

11.6 Screen, navigation, and API behavior mapping

Screen/navigation requirement	API behavior
Splash/session check	Use stored JWT; verify session/profile completion. Return 401 for expired sessions.
Profile setup	Do not mark onboarding complete until required fields and 2–5 photos pass server validation.
Men Home	Query /dashboard/recent ; return women only. Support pagination via page and size .
Women Home	Query /connections/received ; sort by createdAt DESC .
Request detail	Return full non-contact sender profile; expose only Approve action.
Sent tab	Return the full request list with PENDING/APPROVED ; never return recall/cancel actions.
Contact/Telegram	Return partner username/link only for APPROVED ; empty username means the UI shows “Telegram not connected”.
Profile/settings	Return subscription badge/capabilities, photos, and editable profile fields.
Report	Require a reason and return a confirmation-safe response.
Subscription modal	Return exact Standard/Premium/Elite plan data, amount in paisa for Razorpay, and retry-safe payment errors.

11.7 Non-functional and acceptance requirements

Backend release acceptance MUST include:

1. Production HTTPS only; Bearer authentication on every protected endpoint.
2. Passwords stored as strong hashes; never log passwords, OTPs, tokens, or payment secrets.
3. Readable JSON errors for validation, authentication, authorization, missing resources, duplicates, quota exhaustion, and network/payment failures.
4. Pagination for discovery and deterministic newest-first ordering for received requests.
5. Atomic request approval and unique-pair enforcement to prevent duplicate/race-condition requests.
6. Subscription expiry checked on every gated request, not only when the profile screen loads.
7. Multipart image upload returns an absolute CDN URL and the updated image record immediately.
8. All destructive account/report operations remain confirmation-safe on the client and ownership-checked on the server.

The backend team should validate at minimum: registration/OTP, login/session expiry, 18+ validation, 2-photo profile completion, women-only discovery, duplicate request rejection, pending privacy, approval unlock, Telegram persistence after restart, report submission, two-step deletion, subscription activation/expiry, payment cancellation/retry, and remaining-days countdown (the PRD test cases TC-01 through TC-30).

■ 12. Summary Checklist for Backend Developers

1. ☐ **BR-01 Enforcement**: Ensure API returns HTTP **403** if a female user attempts to call **POST /connections/send** or if a request targets a male user.
2. ☐ **BR-02 State Machine**: **POST /connections/send** creates **PENDING**. **PUT /connections/accept** transitions to **APPROVED**.
3. ☐ **BR-03 Telegram Privacy**: **GET /telegram/link** must verify connection status is **APPROVED** before returning the link; otherwise return HTTP **403**.
4. ☐ **BR-04 Feed Privacy**: Women profiles in **/dashboard/recent** must never include **mobile**, **password_hash**, or personal contact details.
5. ☐ **BR-10 Razorpay Verification**: **POST /razorpay/verify** must verify signature **HMAC_SHA256(orderId + "|" + paymentId, SECRET)** before activating subscription.
6. ☐ **ID Consistency**: Use string identifiers (**userId**, **senderId**, **receiverId**, **requestId**) throughout all APIs.

■ 13. Identified Gaps & Production Readiness Checklist (Backend Action Items)

This section explicitly outlines the specific backend implementations required to transition from the standalone frontend mock environment to live production:

13.1 Daily Invitation Quota & Subscription Gate (Gap 1)

- **Endpoint**: **POST /connections/send**
- **Requirement**:
 1. Check if the authenticated male user has an active record in the **subscriptions** table (**status = 'ACTIVE'** and **end_date > CURRENT_TIMESTAMP**).

- If the user has no active subscription, return **403** with **code=SUBSCRIPTION_REQUIRED**.
- If the user has an active plan with a finite **dailyRequestLimit**, count today's invitations (**SELECT COUNT(*) FROM connections WHERE sender_id = :userId AND created_at >= CURRENT_DATE**).
- If count reaches the plan's daily limit (Standard **10**, Premium **20**), return:

```
HTTP 403 Forbidden
{
  "status": "error",
  "message": "Today's invitation limit has been reached. Upgrade your plan to keep connecting.",
  "code": "QUOTA_EXCEEDED"
}
```

- Elite has **dailyRequestLimit=null** and is unlimited. Every request must still pass profile completeness, gender, duplicate, and approval-state checks.

13.2 Live Razorpay Signature Verification (Gap 2)

- **Endpoint:** **POST /razorpay/verify**
- **Requirement:**

- Verify the authenticity of payment using HMAC SHA-256 before activating any subscription:

```
# Python Example
import hmac
import hashlib

generated_signature = hmac.new(
    bytes(RAZORPAY_KEY_SECRET, 'utf-8'),
    bytes(order_id + "|" + payment_id, 'utf-8'),
    hashlib.sha256
).hexdigest()

if generated_signature != razorpay_signature:
    raise InvalidSignatureError("Signature mismatch")
```

- Upon successful validation, insert or update the **subscriptions** record with duration based on selected plan (**BASIC** = 30 days, **GOLD** = 90 days, **PREMIUM** = 180 days).

13.3 Cloud Media Storage & Multipart Upload (Gap 3)

- **Endpoint:** **POST /profile/upload-image**
 - **Requirement:**
- Handle multipart form data (**Content-Type: multipart/form-data**) with field **image / file**.
 - Upload binary payload to AWS S3, Google Cloud Storage, or Cloudflare R2.
 - Return absolute public CDN URL in response:

```
{
  "status": "success",
  "message": "Image uploaded successfully",
  "data": {
    "id": 105,
    "imageUrl": "https://cdn.amara-app.com/photos/usr_woman_201_photo_1.jpg",
    "isProfile": false
  }
}
```